FULL-STACK WEB DEVELOPMENT

# Case Study: Student Management System

**React, Express.js, MongoDB Atlas and Cloud Deployment**

| Course Details | Information |
|---|---|
| Course Title | Full-Stack Web Development |
| Course Code | 23CM4121 |
| Name | M.Niharika |
| Roll No | A24126552097 |
| Case Study | Student Management System |
| Technology Stack | React 18 (Vite), Node.js, Express.js , Mongoose, MongoDB Atlas |
| Hosting | Vercel (frontend), Render (backend), MongoDB Atlas (database) |
| Source Code | github.com/niharikamente/FullStack-LAB/CASE-STUDY/student-management-system |

## PART A. Comprehensive Question

### Case Study: Student Management System

A department office keeps student details in registers and spreadsheets. Staff find it difficult to search for a student, correct a record or add a new admission without duplicating effort. The department wants a web-based Student Management System that stores records in a database and can be reached through a web address.

Design and develop a database-driven Student Management System using React, Node.js, Express.js and MongoDB. Deploy the frontend and backend on suitable cloud platforms and make the application accessible through a web address.

The application must support the following requirements:

- Store student details such as name, email address, course and roll number.
- Create and manage a MongoDB database and students collection using an appropriate document structure.
- Perform CRUD operations: add a student, display students, modify student details and delete a student.
- Write MongoDB queries to insert, retrieve, update and delete documents.
- Use aggregation to generate a course-wise count of students.
- Create indexes to support efficient student lookup.
- Deploy the backend, frontend and database using cloud platforms. Explain the role of hosting, domain names and environment variables.
- Test the deployed application and confirm that data entered through the interface is stored in MongoDB and can be retrieved.

# PART B. Worked Solution

## 1. Analyse the requirements

| Requirement | Solution |
|---|---|
| Student identity | Each record has an automatically generated MongoDB _id; the roll number (for example CS101) is the human-readable identifier. |
| Student details | name, email, course and rollNumber are stored in each document, along with createdAt and updatedAt timestamps. |
| Data storage | MongoDB Atlas database; collection: students (Mongoose pluralises the Student model). |
| Application operations | React sends HTTP requests to Express API routes; Express reads and writes MongoDB documents through Mongoose. |
| Validation | Name and email are required; email is trimmed, lower-cased and matched against a pattern. The API returns HTTP 400 for invalid data. |
| Resilience | If MONGODB_URI is missing or the connection fails, the API serves an in-memory list so the interface still works during development. |
| Search | The React interface filters the loaded list by name, email, course or roll number. |
| Deployment | MongoDB Atlas hosts the database; Render hosts the backend API; Vercel hosts the React frontend. |

## 2. Design the MongoDB document

A student is represented as one BSON document in the students collection. A possible document is:

```
{
  "name": "Aarav Sharma",
  "email": "aarav@example.com",
  "course": "Computer Science",
  "rollNumber": "CS101"
}
```

### Field design:

| Field | Data type | Purpose and rule (from the Mongoose schema) |
|---|---|---|
| name | String | Student full name. Required; trimmed. |
| email | String | Email address. Required; trimmed; stored in lower case; must match /^\S+@\S+\.\S+$/. |
| course | String | Course or programme. Optional; trimmed; defaults to "General". |
| rollNumber | String | Roll number such as CS101. Optional; trimmed; defaults to an empty string. |
| createdAt, updatedAt | Date | Added automatically by the timestamps option. The list is sorted by createdAt, newest first. |

MongoDB also adds an _id field. The Student model converts it to a string field named id in every JSON response and removes __v, so the React code uses student.id when editing or deleting a record.

## 3. Create the database and collection

In MongoDB Shell (mongosh), execute the following commands. The database name is chosen here for illustration; when using Atlas, put the database name at the end of MONGODB_URI (for example .../student_db) so that Mongoose uses it. If no name is given, MongoDB uses the default database named test.

```
use student_db

db.createCollection("students")

db.students.insertMany([
  { name: "Aarav Sharma", email: "aarav@example.com",
    course: "Computer Science", rollNumber: "CS101" },
  { name: "Mira Patel", email: "mira@example.com",
    course: "Information Technology", rollNumber: "IT102" },
  { name: "Niharika Mente", email: "niharika@example.com",
    course: "Computer Science", rollNumber: "CS103" }
])
```

Expected result: the students collection contains three sample documents. The values are illustrative. The backend also seeds the first two records automatically when it connects to an empty collection.

## 4. Perform MongoDB CRUD operations

| Operation | MongoDB query | Purpose |
|---|---|---|
| Create | `db.students.insertOne({ name:"Rahul Verma", email:"rahul@example.com", course:"Electronics", rollNumber:"EC104" })` | Adds a student document. |
| Read | `db.students.find({ course:"Computer Science" })` | Retrieves students of one course. |
| Update | `db.students.updateOne({ rollNumber:"EC104" }, { $set:{ course:"Electronics & Communication" } })` | Changes the course of a student. |
| Delete | `db.students.deleteOne({ rollNumber:"EC104" })` | Removes the matching document. |

Useful additional queries:

```
// Display all students
db.students.find({})

// Find a student by email
db.students.find({ email: "mira@example.com" })

// Sort students by name, A to Z
db.students.find({}).sort({ name: 1 })

// Newest records first (the order used by the API)
db.students.find({}).sort({ createdAt: -1 })

// Count students in the collection
db.students.countDocuments({})
```

## 5. Create indexes

An index helps MongoDB locate documents without scanning the whole collection. The project code does not declare any index beyond the default one on _id, so the following indexes are recommended additions for lookups by email and by course.

```
db.students.createIndex({ email: 1 }, { unique: true })
db.students.createIndex({ course: 1 })

// View indexes
db.students.getIndexes()
```

The unique email index prevents two students from sharing one email address, and the course index supports filtering and the aggregation in the next section. Indexes use storage and slow writes slightly, so they should match the queries the application actually performs. Because the API does not currently check for duplicate emails, creating the unique index would make MongoDB reject a duplicate with an error.

## 6. Use aggregation for a course-wise summary

The department wants to know how many students are enrolled in each course.

```
db.students.aggregate([
  { $group: { _id: "$course", studentCount: { $sum: 1 } } },
  { $sort: { studentCount: -1 } }
])
```

For the three sample records, the expected result is:

| Course | Student count |
|---|---|
| Computer Science | 2 |
| Information Technology | 1 |

The output is grouped by course and sorted by count in descending order. The figures are based on the sample documents above.

## 7. Develop the Express.js backend

The backend lives in the backend folder and uses ES modules ("type": "module"). Initialise the project and install the packages:

```
cd backend
npm install express mongoose cors dotenv
```

backend/package.json:

```
{
  "name": "student-management-api",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "start": "node src/server.js",
    "dev": "node --watch src/server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.7",
    "express": "^4.21.2",
    "mongoose": "^9.12.0"
  }
}
```

backend/src/models/Student.js defines the schema, validation and JSON transform:

```
import mongoose from 'mongoose';

const studentSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Student name is required'],
      trim: true,
    },
    email: {
      type: String,
      required: [true, 'Student email is required'],
      trim: true,
      lowercase: true,
      match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email address'],
    },
    course: {
      type: String,
      trim: true,
      default: 'General',
    },
    rollNumber: {
      type: String,
      trim: true,
      default: '',
    },
  },
  {
    timestamps: true,
    toJSON: {
      virtuals: true,
      transform: (_doc, ret) => {
```

```
        ret.id = ret._id.toString();
        delete ret._id;
        delete ret.__v;
        return ret;
      },
    },
  }
);

export const Student = mongoose.model('Student', studentSchema);
```

backend/src/server.js configures CORS, connects to MongoDB Atlas, seeds sample data and exposes the REST API:

```
import dotenv from 'dotenv';
import path from 'path';
import { fileURLToPath } from 'url';
import dns from 'dns';
import express from 'express';
import cors from 'cors';
import mongoose from 'mongoose';
import { Student } from './models/Student.js';

// Resolve DNS SRV query issues on Windows
try {
  dns.setServers(['8.8.8.8', '1.1.1.1']);
} catch {
  // Use default system DNS if override fails
}

// Ensure .env is loaded regardless of execution directory
const __dirname = path.dirname(fileURLToPath(import.meta.url));
dotenv.config({ path: path.resolve(  dirname, '../.env') });
dotenv.config({ path: path.resolve(process.cwd(), '.env') });

const app = express();
const port = process.env.PORT || 5000;

const allowedOrigins = [
  process.env.FRONTEND_ORIGIN,
  'http://localhost:5173',
  'http://127.0.0.1:5173',
  'http://localhost:5174',
  'http://127.0.0.1:5174',
].filter(Boolean);

app.use(
  cors({
    origin: (origin, callback) => {
      if (!origin || allowedOrigins.includes(origin) || process.env.NODE_ENV !== 'production') {
        return callback(null, true);
      }
      return callback(new Error('Not allowed by CORS'));
    },
  })
);

app.use(express.json());

// In-memory fallback if MongoDB is not connected
const memoryStudents = [
  { id: '1', name: 'Aarav Sharma', email: 'aarav@example.com', course: 'Computer Science',
rollNumber: 'CS101' },
  { id: '2', name: 'Mira Patel', email: 'mira@example.com', course: 'Information Technology',
rollNumber: 'IT102' },
];

let isDbConnected = false;

// Connect to MongoDB Atlas
const connectDB = async () => {
  const uri = process.env.MONGODB_URI;
  if (!uri) {
    console.warn('⚠️ MONGODB_URI not found in environment variables. Using in-memory fallback.');
    return;
  }

  try {
```

```
    await mongoose.connect(uri);
    isDbConnected = true;
    console.log('✅ Connected to MongoDB Atlas successfully');

    // Seed default students if collection is empty
    const count = await Student.countDocuments();
    if (count === 0) {
      await Student.insertMany([
        { name: 'Aarav Sharma', email: 'aarav@example.com', course: 'Computer Science',
rollNumber: 'CS101' },
        { name: 'Mira Patel', email: 'mira@example.com', course: 'Information Technology',
rollNumber: 'IT102' },
      ]);
      console.log('🌱 Seeded sample students into MongoDB Atlas');
    }
  } catch (error) {
    console.error('❌ MongoDB Connection Error:', error.message);
    console.error('💡 Tip: Ensure your current IP is whitelisted in MongoDB Atlas Network Access
(or set to 0.0.0.0/0).');
  }
};

mongoose.connection.on('disconnected', () => {
  isDbConnected = false;
  console.warn('⚠️ MongoDB disconnected.');
});

mongoose.connection.on('connected', () => {
  isDbConnected = true;
});

// Health check
app.get('/api/health', (_req, res) => {
  res.json({
    ok: true,
    database: isDbConnected ? 'MongoDB Atlas' : 'in-memory-fallback',
  });
});

// GET /api/students - List all students
app.get('/api/students', async (_req, res) => {
  try {
    if (isDbConnected) {
      const students = await Student.find().sort({ createdAt: -1 });
      return res.json(students);
    }
    return res.json(memoryStudents);
  } catch (error) {
    return res.status(500).json({ error: error.message });
  }
});

// POST /api/students - Add a student
app.post('/api/students', async (req, res) => {
  const { name, email, course, rollNumber } = req.body;
  if (!name || !email) {
    return res.status(400).json({ error: 'Name and email are required.' });
  }

  try {
    if (isDbConnected) {
      const newStudent = await Student.create({
        name,
        email,
        course: course || 'General',
        rollNumber: rollNumber || '',
      });
      return res.status(201).json(newStudent);
    }

    // In-memory fallback
    const newStudent = {
      id: String(Date.now()),
      name,
      email,
      course: course || 'General',
      rollNumber: rollNumber || '',
    };
    memoryStudents.unshift(newStudent);
```

```
    return res.status(201).json(newStudent);
  } catch (error) {
    return res.status(400).json({ error: error.message });
  }
});

// PUT /api/students/:id - Update student
app.put('/api/students/:id', async (req, res) => {
  const { id } = req.params;
  const { name, email, course, rollNumber } = req.body;

  try {
    if (isDbConnected) {
      const updated = await Student.findByIdAndUpdate(
        id,
        { name, email, course, rollNumber },
        { new: true, runValidators: true }
      );
      if (!updated) return res.status(404).json({ error: 'Student not found.' });
      return res.json(updated);
    }

    const index = memoryStudents.findIndex((s) => s.id === id);
    if (index === -1) return res.status(404).json({ error: 'Student not found.' });
    memoryStudents[index] = { ...memoryStudents[index], name, email, course, rollNumber };
    return res.json(memoryStudents[index]);
  } catch (error) {
    return res.status(400).json({ error: error.message });
  }
});

// DELETE /api/students/:id - Delete student
app.delete('/api/students/:id', async (req, res) => {
  const { id } = req.params;

  try {
    if (isDbConnected) {
      const deleted = await Student.findByIdAndDelete(id);
      if (!deleted) return res.status(404).json({ error: 'Student not found.' });
      return res.json({ message: 'Student deleted successfully.', id });
    }

    const index = memoryStudents.findIndex((s) => s.id === id);
    if (index === -1) return res.status(404).json({ error: 'Student not found.' });
    const [deleted] = memoryStudents.splice(index, 1);
    return res.json({ message: 'Student deleted successfully.', id: deleted.id });
  } catch (error) {
    return res.status(500).json({ error: error.message });
  }
});

connectDB();

app.listen(port, () => {
  console.log(`Student API listening on port ${port}`);
});
```

backend/.env.example (use your own values in a private .env file):

```
PORT=5000
FRONTEND_ORIGIN=http://localhost:5173
# Future MongoDB Atlas connection string. Keep real credentials in an untracked .env file.
MONGODB_URI=mongodb+srv://<username>:<password>@cluster0.example.mongodb.net/
```

## API endpoint summary:

| Method | Endpoint | Action |
|---|---|---|
| GET | /api/health | Reports ok: true and whether the data source is MongoDB Atlas or the in-memory fallback. |
| GET | /api/students | Retrieves all students, newest first. |
| POST | /api/students | Creates a student; returns 201, or 400 if name or email is missing. |
| PUT | /api/students/:id | Updates a student by id; returns 404 if not found. |

| Method | Endpoint | Action |
|---|---|---|
| DELETE | /api/students/:id | Deletes a student by id; returns 404 if not found. |

### Design notes:

- **CORS.** The API allows FRONTEND_ORIGIN and the local Vite addresses. When NODE_ENV is not "production" every origin is allowed, so set NODE_ENV=production on Render to enforce the list.
- **Fallback mode.** The isDbConnected flag switches every route between Mongoose and the in-memory array, and mongoose connection events keep the flag current.
- **DNS setting.** server.js sets public DNS servers (8.8.8.8, 1.1.1.1) to avoid SRV lookup failures for mongodb+srv addresses on some Windows networks.

## 8. Develop the React frontend

The frontend is a Vite React application in the frontend folder. Install and start it with:

```
cd frontend
npm install
npm run dev
```

Set the API base URL in frontend/.env (see .env.example):

```
VITE_API_URL=http://localhost:5000
```

frontend/src/App.jsx is the single component of the application. It shows the database status badge, an add/edit form, a search box and the student list with Edit and Delete buttons:

```
import React, { useEffect, useState } from 'react';

const API_URL = (import.meta.env.VITE_API_URL || 'http://localhost:5000').replace(/\/$/, '');

export default function App() {
  const [students, setStudents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [dbStatus, setDbStatus] = useState('Connecting...');
  const [searchTerm, setSearchTerm] = useState('');

  // Form state
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    course: '',
    rollNumber: '',
  });

  // Editing state
  const [editingId, setEditingId] = useState(null);

  const fetchStudents = async () => {
    try {
      const response = await fetch(`${API_URL}/api/students`);
      if (!response.ok) throw new Error(`API returned ${response.status}`);
      const data = await response.json();
      setStudents(data);
      setError('');
    } catch {
      setError('Could not load students. Check that the backend API is running on port 5000.');
    } finally {
      setLoading(false);
    }
  };

  const fetchHealth = async () => {
    try {
      const response = await fetch(`${API URL}/api/health`);
      if (response.ok) {
        const data = await response.json();
        setDbStatus(data.database || 'Connected');
      }
    } catch {
      setDbStatus('Offline');
    }
```

```
};

useEffect(() => {
  fetchStudents();
  fetchHealth();
}, []);

const handleInputChange = (e) => {
  const { name, value } = e.target;
  setFormData((prev) => ({ ...prev, [name]: value }));
};

const handleSubmit = async (e) => {
  e.preventDefault();
  if (!formData.name.trim() || !formData.email.trim()) {
    alert('Name and Email are required.');
    return;
  }

  try {
    if (editingId) {
      // Update existing student
      const response = await fetch(`${API_URL}/api/students/${editingId}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(formData),
      });
      if (!response.ok) throw new Error('Failed to update student');
      const updated = await response.json();
      setStudents((prev) => prev.map((s) => (s.id === editingId ? updated : s)));
      setEditingId(null);
    } else {
      // Create new student in MongoDB
      const response = await fetch(`${API_URL}/api/students`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(formData),
      });
      if (!response.ok) throw new Error('Failed to create student');
      const created = await response.json();
      setStudents((prev) => [created, ...prev]);
    }

    setFormData({ name: '', email: '', course: '', rollNumber: '' });
    setError('');
  } catch (err) {
    alert(err.message || 'Operation failed');
  }
};

const handleEdit = (student) => {
  setEditingId(student.id);
  setFormData({
    name: student.name || '',
    email: student.email || '',
    course: student.course || '',
    rollNumber: student.rollNumber || '',
  });
  window.scrollTo({ top: 0, behavior: 'smooth' });
};

const handleCancelEdit = () => {
  setEditingId(null);
  setFormData({ name: '', email: '', course: '', rollNumber: '' });
};

const handleDelete = async (id) => {
  if (!window.confirm('Are you sure you want to delete this student record?')) return;

  try {
    const response = await fetch(`${API_URL}/api/students/${id}`, {
      method: 'DELETE',
    });
    if (!response.ok) throw new Error('Failed to delete student');
    setStudents((prev) => prev.filter((s) => s.id !== id));
  } catch (err) {
    alert(err.message || 'Delete failed');
  }
};
```

```
const filteredStudents = students.filter((s) => {
  const term = searchTerm.toLowerCase();
  return (
    (s.name && s.name.toLowerCase().includes(term)) ||
    (s.email && s.email.toLowerCase().includes(term)) ||
    (s.course && s.course.toLowerCase().includes(term)) ||
    (s.rollNumber && s.rollNumber.toLowerCase().includes(term))
  );
});

return (
  <main>
    <div className="header-row">
      <div>
        <p className="eyebrow">Student Management System</p>
        <h1>Students</h1>
        <p className="intro">Manage records with MongoDB Atlas cloud persistence.</p>
      </div>
      <div className="badge">
        <span className="badge-dot"></span>
        <span>Database: {dbStatus}</span>
      </div>
    </div>

    {error && (
      <div className="error-banner" role="alert">
        {error}
      </div>
    )}

    {/* Add / Edit Form */}
    <div className="form-card">
      <h2>{editingId ? 'Edit Student Record' : 'Add New Student'}</h2>
      <form onSubmit={handleSubmit}>
        <div className="form-grid">
          <input
            type="text"
            name="name"
            placeholder="Full Name *"
            value={formData.name}
            onChange={handleInputChange}
            required
          />
          <input
            type="email"
            name="email"
            placeholder="Email Address *"
            value={formData.email}
            onChange={handleInputChange}
            required
          />
          <input
            type="text"
            name="rollNumber"
            placeholder="Roll Number (e.g. CS101)"
            value={formData.rollNumber}
            onChange={handleInputChange}
          />
          <input
            type="text"
            name="course"
            placeholder="Course (e.g. Computer Science)"
            value={formData.course}
            onChange={handleInputChange}
          />
        </div>
        <div style={{ display: 'flex', gap: '0.5rem' }}>
          <button type="submit" className="btn btn-primary">
            {editingId ? 'Save Changes' : '+ Add Student to MongoDB'}
          </button>
          {editingId && (
            <button type="button" className="btn btn-secondary" onClick={handleCancelEdit}>
              Cancel
            </button>
          )}
        </div>
      </form>
    </div>

    {/* Toolbar */}
```

```
      <div className="toolbar">
        <input
          type="text"
          className="search-input"
          placeholder="🔍 Search by name, email, roll no..."
          value={searchTerm}
          onChange={(e) => setSearchTerm(e.target.value)}
        />
        <span className="students-count">
          Showing {filteredStudents.length} of {students.length} students
        </span>
      </div>

      {/* Student List */}
      {loading ? (
        <p className="empty-state">Loading students from MongoDB...</p>
      ) : filteredStudents.length === 0 ? (
        <div className="empty-state">
          {searchTerm ? 'No students matched your search.' : 'No students found. Add your first
student above!'}
        </div>
      ) : (
        <ul className="student-list">
          {filteredStudents.map((student) => (
            <li key={student.id} className="student-item">
              <div className="student-info">
                <div className="student-main-line">
                  <span className="student-name">{student.name || 'Student (No Name)'}</span>
                  {student.rollNumber && <span className="tag-roll">{student.rollNumber}</span>}
                  {student.course && <span className="tag-course">{student.course}</span>}
                </div>
                <div className="student-sub-line">
                  <span>{student.email || 'No email provided'}</span>
                </div>
              </div>
              <div className="student-actions">
                <button
                  type="button"
                  className="btn btn-edit"
                  onClick={() => handleEdit(student)}
                >
                  Edit
                </button>
                <button
                  type="button"
                  className="btn btn-danger"
                  onClick={() => handleDelete(student.id)}
                >
                  Delete
                </button>
              </div>
            </li>
          ))}
        </ul>
      )}
    </main>
  );
}
```

### Frontend features:

- Reads the backend address from VITE_API_URL, so the same build logic works locally and after deployment.
- Loads students and the health status when the page opens and shows an error banner if the API cannot be reached.
- Uses one form for both adding and editing; choosing Edit fills the form and changes the button to Save Changes.
- Asks for confirmation before deleting a record.
- Filters the list by name, email, course or roll number and displays "Showing X of Y students".

## 9. Run and test the application locally

1. Create backend/.env from .env.example and set MONGODB_URI to your MongoDB Atlas (or local MongoDB) connection string. Without it the API runs in-memory mode.

2. From the backend folder, run npm install and then npm start (or npm run dev). The console should print "Connected to MongoDB Atlas successfully".
3. Create frontend/.env with VITE_API_URL=http://localhost:5000. From the frontend folder, run npm install and npm run dev, then open the Vite URL (normally http://localhost:5173).
4. Add a student through the form. Confirm that it appears in the list and in the students collection.
5. Edit the student and confirm that the changed value is displayed and stored.
6. Delete a test student and verify that it is removed from the collection.
7. Open http://localhost:5000/api/health and check the health response.

The repository root also provides npm run install:all and npm run dev, which install both folders and start the API and the frontend together.

| Test case | Action | Expected result |
|---|---|---|
| TC1: Create | Submit the form with valid name and email. | New student is stored and displayed at the top of the list. |
| TC2: Read | Open or reload the application. | Stored students are retrieved from the API and listed. |
| TC3: Update | Edit the course of a student and save. | Updated value is visible in the list and after refresh. |
| TC4: Delete | Delete a test student and confirm. | Student is removed from the list and the database. |
| TC5: Validation | POST a record without an email. | API rejects it with HTTP 400 and an error message. |
| TC6: Search | Type "CS" in the search box. | Only matching students are shown, with an updated count. |

## 10. Sample execution results

The application was run locally and exercised through the browser and the API. These runs used the in-memory fallback because no MongoDB connection string was configured, which is why the status badge reads "in-memory-fallback". Repeat them with MONGODB_URI set to capture evidence of data stored in MongoDB Atlas.

| Test case | Observed result | Status |
|---|---|---|
| TC1: Create | POST returned the new record with an id; it appeared at the top of the list (Screens 2 and 7). | Passed |
| TC2: Read | GET /api/students returned all records, newest first. | Passed |
| TC3: Update | PUT changed the course of an existing record; the list showed the new value (Screen 3). | Passed |
| TC4: Delete | DELETE returned HTTP 200; a repeat call returned 404 "Student not found." | Passed |
| TC5: Validation | POST without email returned HTTP 400 "Name and email are required." | Passed |
| TC6: Search | Searching "CS" showed 3 of 4 students (Screen 4). | Passed |

## Useful API test commands

Run these commands in a terminal after the backend starts. Replace <id> with the id returned by the API.

```
# Health check
curl http://localhost:5000/api/health
```

```
# Read students
curl http://localhost:5000/api/students

# Create student (Git Bash / macOS / Linux)
curl -X POST http://localhost:5000/api/students \
  -H 'Content-Type: application/json' \
  -d '{"name":"Karan
Rao","email":"karan@example.com","course":"Mechanical","rollNumber":"ME105"}'

# Update student (replace <id>)
curl -X PUT http://localhost:5000/api/students/<id> \
  -H 'Content-Type: application/json' \
  -d '{"name":"Karan
Rao","email":"karan@example.com","course":"Civil","rollNumber":"ME105"}'

# Delete student (replace <id>)
curl -X DELETE http://localhost:5000/api/students/<id>
```

Note: the PUT route replaces name, email, course and rollNumber together, so send all four fields.

## 11. Deploy the application using cloud platforms

Deployment architecture:

| Layer | Platform | Responsibility |
|---|---|---|
| Frontend | Vercel | Builds and hosts the React application and provides a public frontend URL. vercel.json rewrites all paths to index.html. |
| Backend | Render | Runs the Node.js/Express API and provides a public API URL. |
| Database | MongoDB Atlas | Hosts the MongoDB database and stores student documents. |

### Deployment steps:

1. Create a MongoDB Atlas cluster. Create a database user, configure network access according to the classroom or production security requirements, and copy the connection string, adding the database name (for example student_db). Use a strong password and do not share it.
2. Push the project to a GitHub repository with the frontend and backend folders. Do not commit .env files or credentials (the repository .gitignore excludes them).
3. On Render, create a Web Service from the repository and set the root directory to backend. Set the build command to npm install and the start command to npm start.
4. Add backend environment variables in Render: MONGODB_URI, FRONTEND_ORIGIN=<your Vercel frontend URL> and NODE_ENV=production. Render supplies PORT itself.
5. Deploy the backend and open its /api/health endpoint. It should return ok: true and database: MongoDB Atlas. Copy the public backend URL.
6. On Vercel, import the repository, set the root directory to frontend and the framework preset to Vite. Set VITE_API_URL to the Render backend URL (without a trailing slash) and deploy.
7. Update FRONTEND_ORIGIN in Render with the final Vercel URL and redeploy the backend if necessary. Test create, read, update and delete from the public frontend.
8. For a custom domain, register or use a domain and add the DNS records that Vercel (or Render) requests. Verify the domain after DNS changes have propagated.

### Role of hosting, domain names and environment variables:

- **Hosting** keeps the application running on a server that is always online, so users do not depend on your laptop.
- **Domain names** give the application a readable address; DNS records map the name to the host. Vercel and Render also provide free default addresses.

- **Environment variables** hold values that differ between environments or must stay secret, such as the database connection string, without writing them in source code.

### Environment variable summary:

| Variable | Where to set it | Example / value |
|---|---|---|
| `MONGODB_URI` | Render backend service | MongoDB Atlas connection string (secret). |
| `FRONTEND_ORIGIN` | Render backend service | https://your-app.vercel.app |
| `NODE_ENV` | Render backend service | production (turns on the CORS origin check). |
| `PORT` | Render (automatic) | Supplied by the host; defaults to 5000 locally. |
| `VITE_API_URL` | Vercel frontend project | https://your-api.onrender.com |

## 12. Expected final outcome

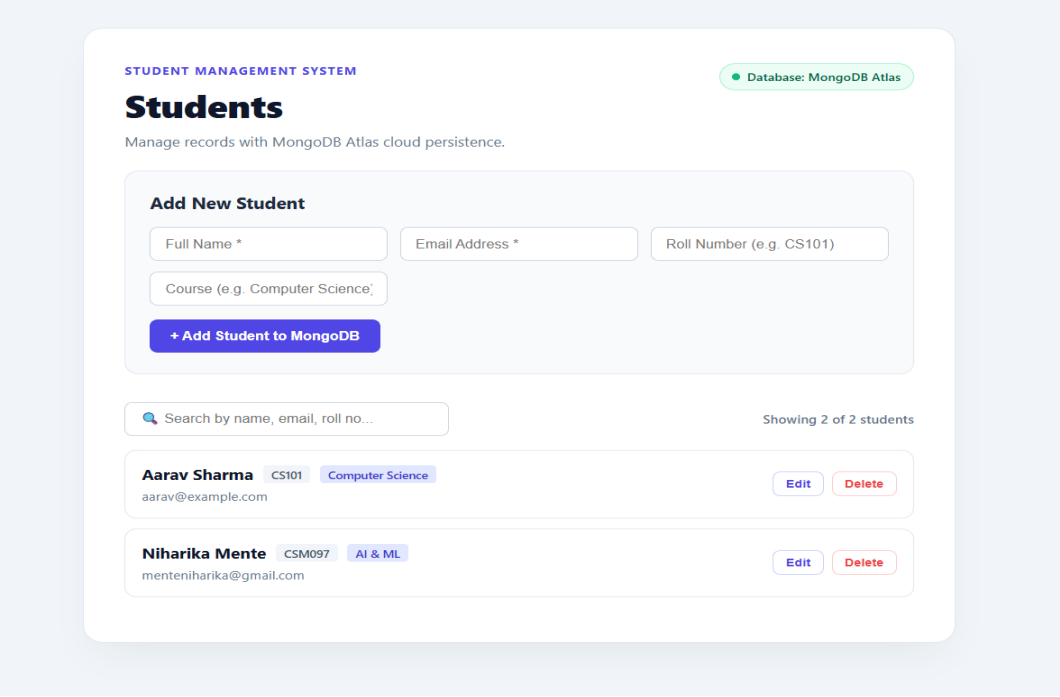
- A public frontend URL that opens the Student Management System.
- A backend API URL that responds to the health-check endpoint.
- A MongoDB Atlas database containing student documents.
- Working add, view, edit, delete and search operations from the user interface.
- A course-wise aggregation and the recommended indexes.
- Evidence that the deployed frontend communicates with the deployed backend and database.

# PART C. Output Screen Shots (Test Case Results)

The screens below were captured from a local run of the project.

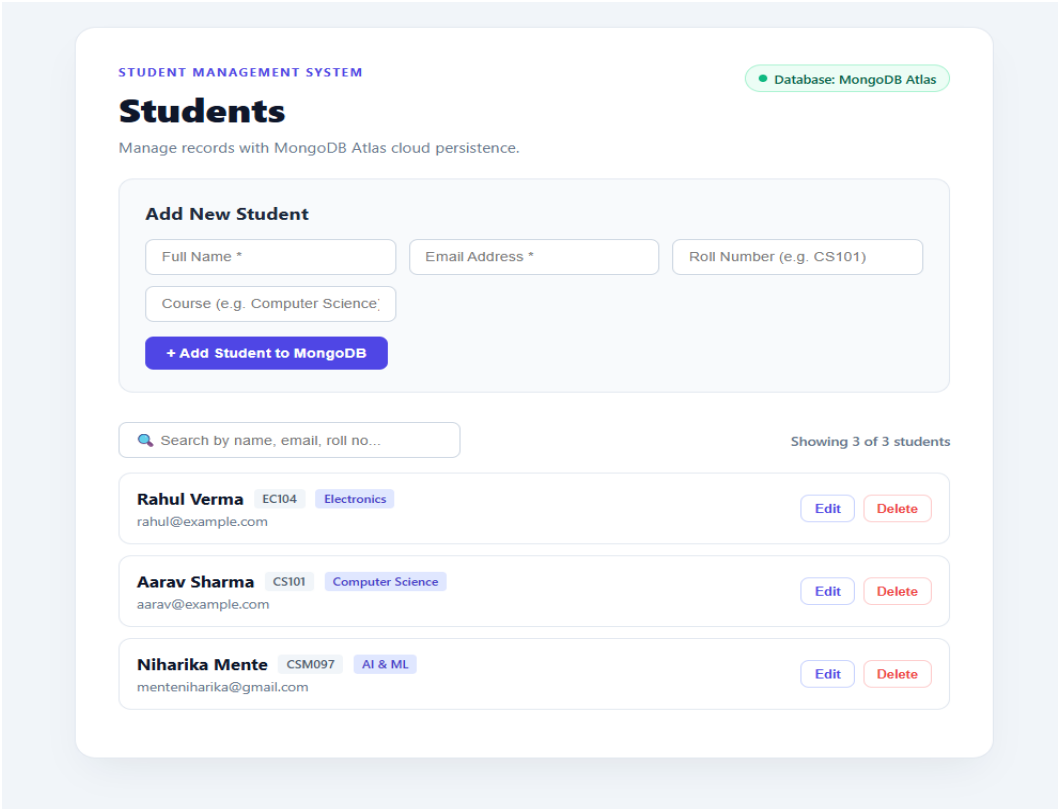
### Screen 1: Student Management Dashboard

Shows the database status badge, the add form, the search box and the seeded student list with Edit and Delete options.



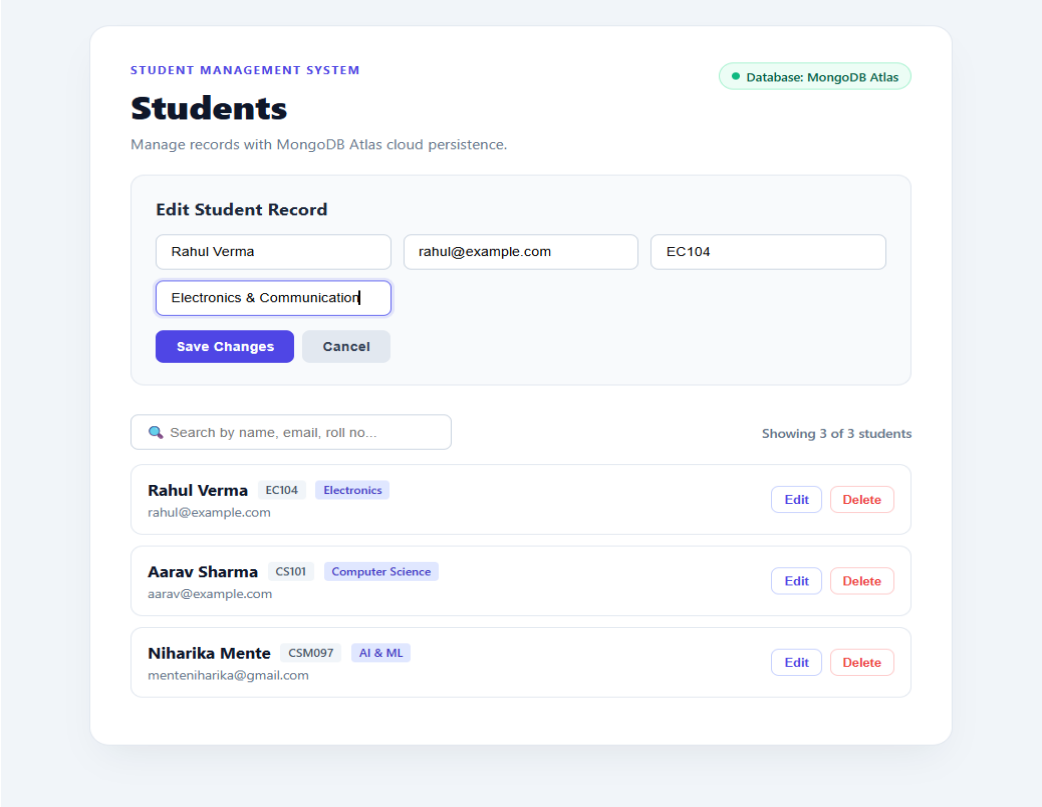

### Screen 2: Student added (TC1)

A new student, Rahul Verma (EC104), appears at the top of the list and the count rises to 3.



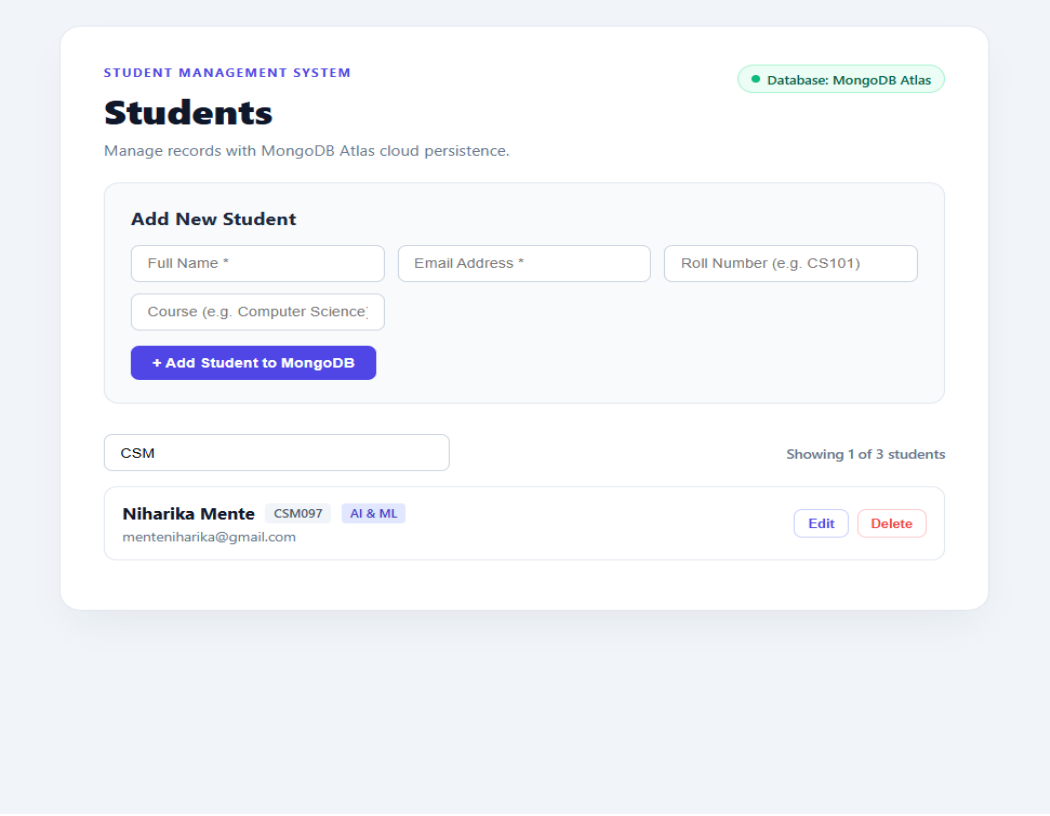
### Screen 3: Editing a record (TC3)

Choosing Edit loads the record into the form, changes the heading to "Edit Student Record" and shows Save Changes and Cancel buttons.



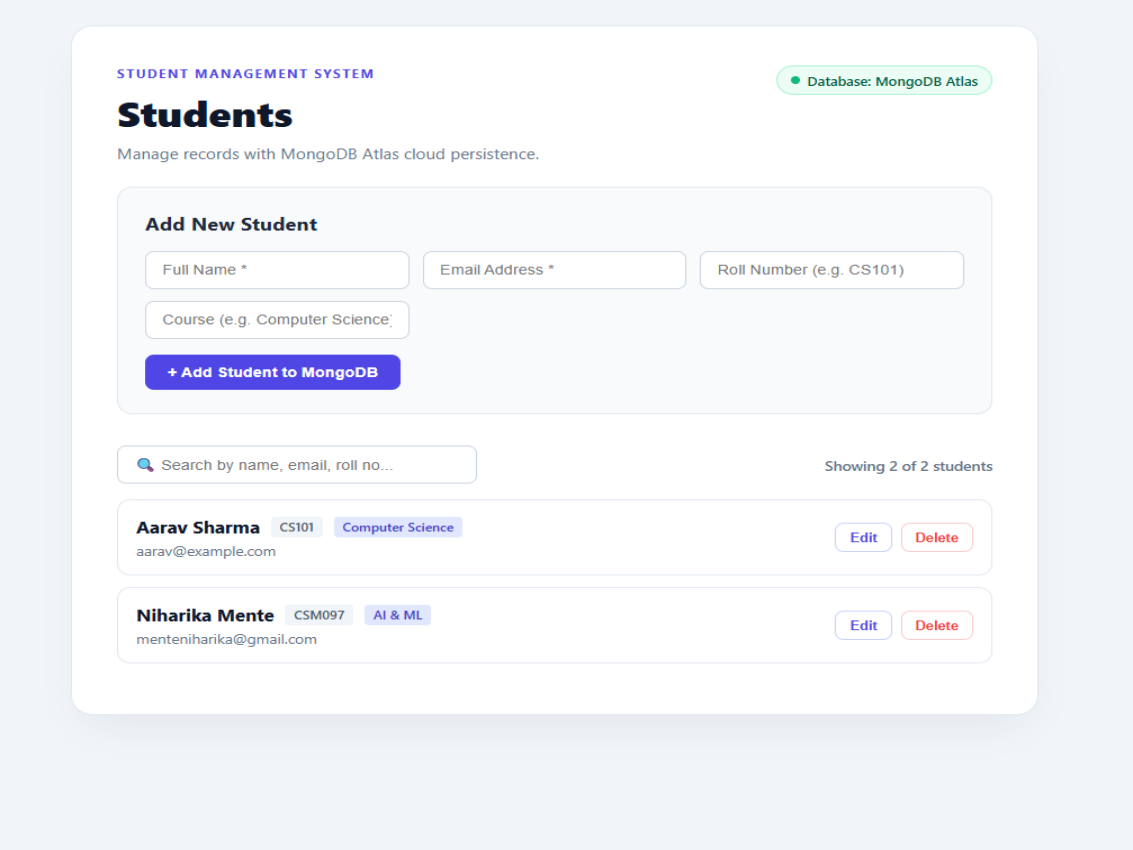

### Screen 4: Search filter (TC6)

Typing "CSM" in the search box narrows the list and shows "Showing 1 of 3 students".



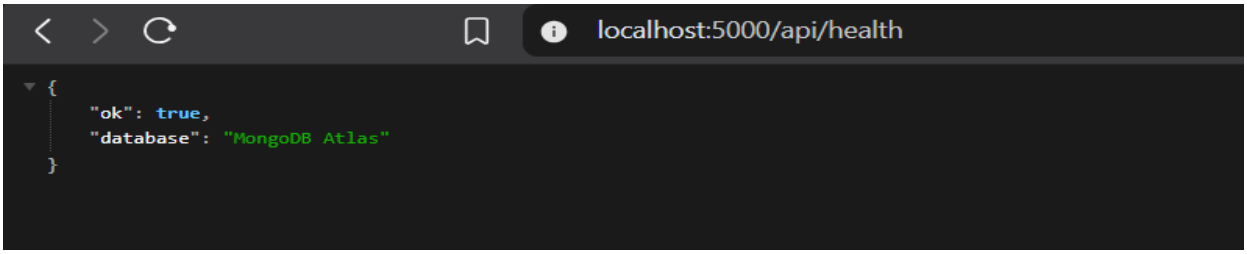
**Screen 5: Student deleted (TC4)**

After confirming deletion the record is removed and the count returns to 2.

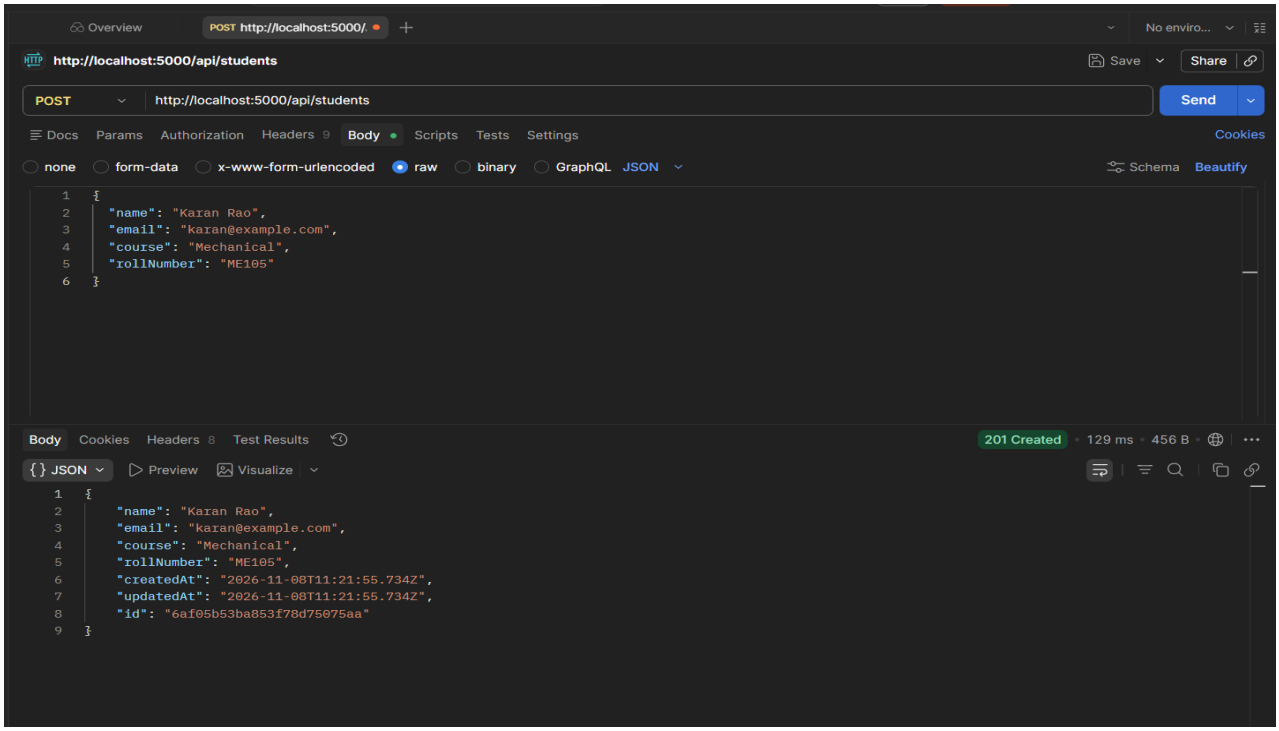



**Screen 6: Backend API health check**

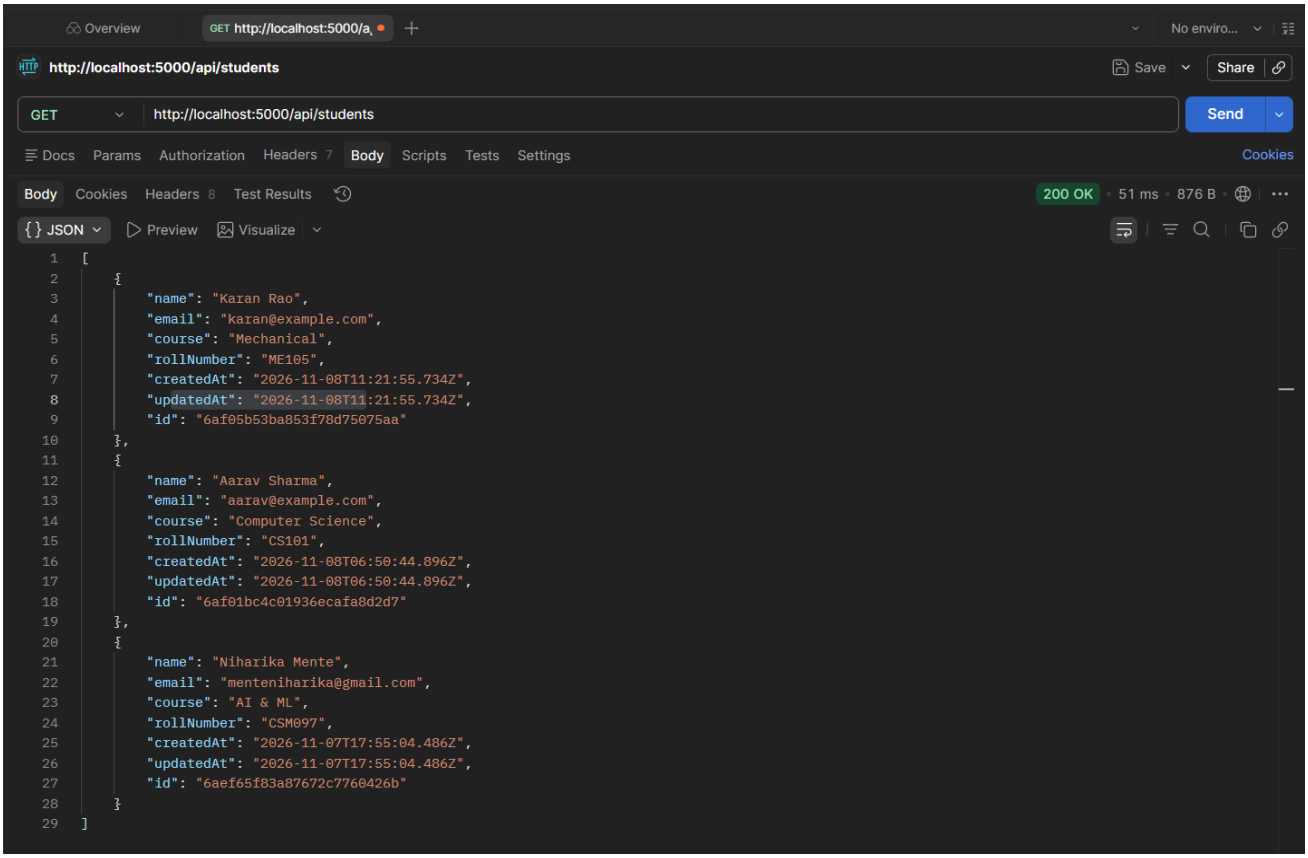
Shows the response from /api/health: ok is true and the data source is reported.



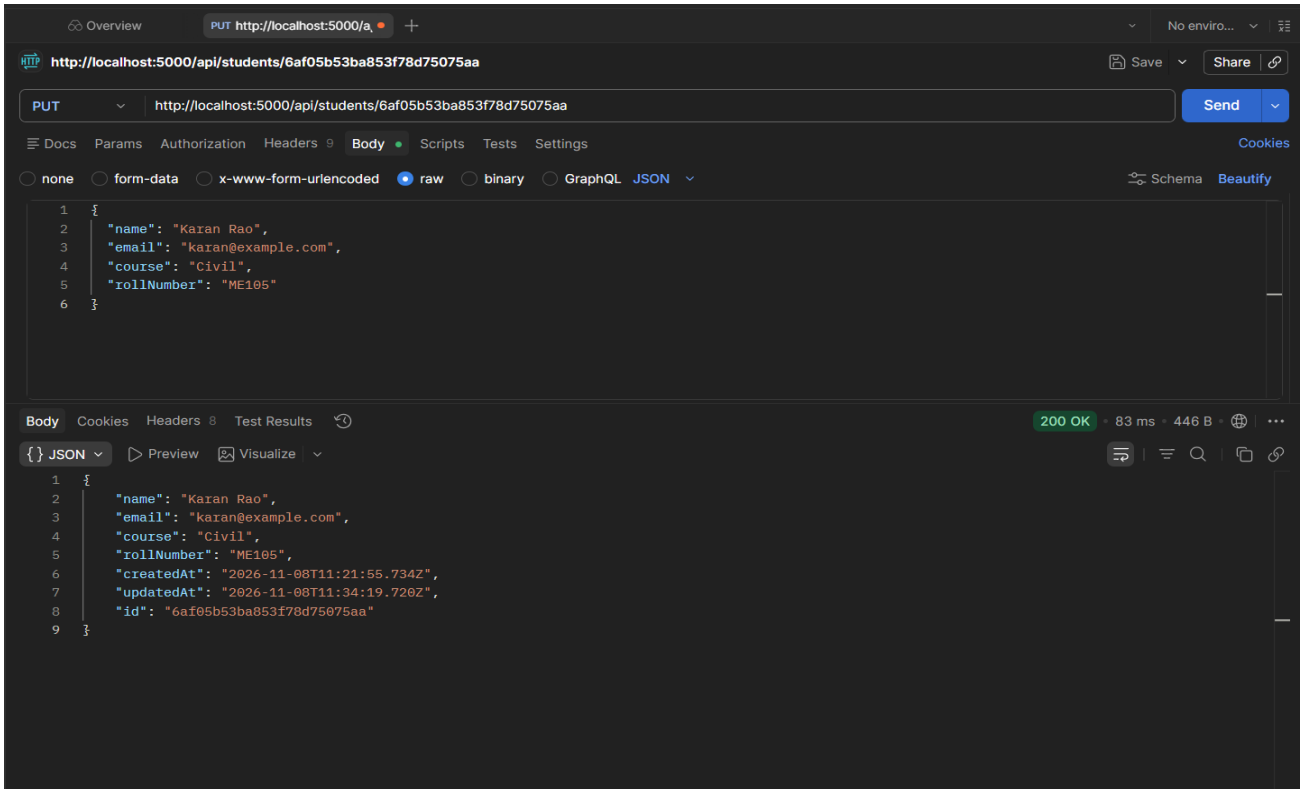
**Screen 7: CRUD and validation API test (TC1 )** POST returned the new record with an id; it appeared at the top of the list




**Screen 8: CRUD and validation API test(TC2)** GET /api/students returned all records, newest first.



**Screen 9: CRUD and validation API test(TC3)** PUT changed the course of an existing record; the list showed the new value




**Screen 10: CRUD and validation API test(TC4)** DELETE returned HTTP 200; a repeat call returned 404 "Student not found."

